Sequential Task Scheduling for Mobile Edge Computing Using Genetic Algorithm

Ahmed A. Al-habob*, Octavia A. Dobre* and Ana García Armada†
*Faculty of Engineering and Applied Science, Memorial University, St. John's, Canada
Email: {*aaaalhabob, odobre*}@mun.ca
†Department of Signal Theory and Communications, University Carlos III de Madrid, Leganés, Spain
Email: {*agarcia*}@ing.uc3m.es

***Abstract*—In this paper, we consider sequential task offloading to multiple mobile-edge computing servers to providing ultra-reliable low-latency mobile edge computing. The task consists of a set of sub-tasks, with a general dependency model among sub-tasks. Our objective is to minimize both latency and offloading failure probability by scheduling sub-tasks to servers. We formulate an optimization problem with constraints over binary scheduling decision variables. A genetic algorithm is devised to solve the formulated optimization problems. Simulation results show that the proposed algorithm provides performance close to the optimal solution, which is obtained through exhaustive search.**

***Index Terms*—Mobile edge computing, sequential task offloading, genetic algorithms.**

I. Introduction

In recent years, a tremendous growth of computationally-intensive mobile applications such as 3D modeling, online gaming, and mobile augmented reality have been witnessed. However, mobile devices (such as smart-watches, tablets and smartphones) are normally constrained by limited resources (e.g., computation capability of local CPUs and capacity-limited battery), and thus, restrict the users ability to run applications that require intensive processing on their own devices. Mobile cloud computing (MCC) has emerged as a solution to provide elastic computing power to resource-constrained devices via offloading the computational-intensive tasks to powerful distant centralized servers [1]. However, locating the serves far away from the end-user has limitations such as high transmission latency, communication bottlenecks, and security issues (e.g., some data should not be offloaded to servers that are located outside of national territory). Thus, MCC is not appropriate for mobile applications that are latency or security critical [2]. This motivated the idea of moving the function of clouds towards the network edges and gave birth to a new computing infrastructure, namely mobile-edge computing (MEC) [3]. MEC provides lower offloading latency when compared to MCC. Furthermore, while MCC is a centralized computation offloading approach, MEC provides a distributed approach which makes it a suitable solution for offloading computational-intensive and time-sensitive mobile applications [4].

In MEC, based on local CPU availability or energy consumption consideration, the mobile device performs the task computation or offloads the task to the MEC servers. This decision plays a critical role in determining the computation efficiency, especially as the task offloading requires data transmission over the wireless channel. Two main computation task offloading policies can be found in the literature, namely partial offloading and binary offloading. In the partial offloading policy, a portion of the task computation is performed at the mobile device and the other portion is performed at the MEC servers [5]. In the binary offloading policy, the task is either performed at the mobile device or offloaded to the MEC servers [6]. From the user point of view, the most important merits of task offloading are the energy consumption, as well as end-to-end latency and reliability, mandating energy efficient and ultra-reliable low-latency task offloading.

Migrating the computation servers towards the network edges enables MEC servers be placed on ultra-dense edge devices, such as micro data centers, wireless access points, small-cell base stations, WiFi routers, and gateways at home. With the dense deployment of MEC servers in future networks, offloading a computational task to multiple nearby MEC servers can potentially improve the offloading process. In [7], a task is divided into sub-tasks and offloaded in parallel to different MEC servers via orthogonal sub-channels. The authors aim to minimize both latency and mobile device's energy consumption. A sequential task offloading framework was proposed in [8]. In this framework, a mobile device segments a task into sub-tasks and offloads them sequentially to multiple MEC servers via a shared communication channel.

In this paper, a delay-sensitive and computationally-intensive task is offloaded to multiple MEC servers. The task consists of a set of sub-tasks, and a general dependency among sub-tasks is considered in the system model. We investigate the sequential offloading scheme and formulate a problem which jointly optimizes the latency and offloading failure probability. This problem is constrained over binary scheduling decision variables, and is NP-hard in nature; accordingly, the optimal solution is not attainable in polynomial time. Consequently, a solution based on genetic algorithm (GA) is designed for the formulated optimization problem.

The remaining of this paper is organized as follows. Section II introduces the system model, along with the sequential offloading scheme. Section III illustrates an example to motivate the need for an optimized scheduling decision. In Section IV, the latency-reliability minimization problem is formulated.

Section V presents the proposed GA-based solution to find the optimized scheduling decision. The time complexity of the solution is discussed in Section VI. Section VII shows simulation results, and Section VIII concludes the paper.

II. System Model and Offloading Scheme

A. System Model

A delay-sensitive and computationally-intensive task ($\mathcal{T}$) is offloaded by a mobile device to a set $\mathcal{S} = \{s_i\}_{i=1}^{I}$ of I MEC servers. Each server is equipped with a CPU that helps offloading and computing the mobile device's task.

A server s_i is represented by a tuple $s_i = \{Ru_i, Rd_i, f_i, p_i, q_i\}$, where Ru_i and Rd_i are the uplink and downlink data rates, respectively, f_i is the computational speed of the CPU in the i-th server (in cycles per second), and p_i and q_i are the packet error rate (PER) of the uplink and downlink, respectively. The size of the input and output computed result of $\mathcal{T}$ are U and D packets, respectively, each packet of N_p bits. The task $\mathcal{T}$ consist of J $(J \leq I)$ sub-tasks $\mathcal{T} = \{\tau_j\}_{j=1}^{J}$. We use a tuple $\tau_j = \{u_j, c_j, d_j\}$ to represent the j-th sub-task in which u_j is the size of the input data (in packets), c_j is the number of CPU cycles required to compute τ_j, and d_j is the output computed result (in packets). c_j is modeled as $c_j = \alpha_j N_p u_j$, where α_j (in cycles per bit) depends on the sub-task's computational complexity [9]. The uplink transmission delay and the computation delay of offloading the sub-task τ_j to the i-th server are $du_{ij} = \frac{u_j N_p}{Ru_i}$ and $dc_{ij} = \frac{c_j}{f_i}$, respectively. We define the offloading decision $\boldsymbol{\eta} = \left[\eta_{ij}\right]_{I\times J}$ such that:

$$\eta_{ij} = \begin{cases} 1, & \text{if } \tau_j \text{ is assigned to } s_i, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Based on the offloading decision $\boldsymbol{\eta}$, the uplink transmission delay and the computation delay of the $j-$th sub-task τ_j can be expressed as $\mathcal{D}u_j(\boldsymbol{\eta}) = \sum_{i=1}^{I} \eta_{ij} du_{ij}$ and $\mathcal{D}c_j(\boldsymbol{\eta}) = \sum_{i=1}^{I} \eta_{ij} dc_{ij}$, respectively.

The dependency among the sub-tasks cannot be ignored in many applications, as it has significant effect on the offloading and computation procedure. We consider the general dependency model in which the computation of a sub-task τ_l may depend on the output computed result of one or more of the previous sub-tasks [10]. To address the inter-dependency among the sub-tasks, we introduce the sub-task dependency matrix $\mathbf{x} = \left[x_{lj}\right]_{J\times J}$ such that:

$$x_{lj} = \begin{cases} 1, & \text{if } \tau_j \text{ depends on the result of } \tau_l\,(l<j) \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

The backhaul links among the servers are considered identical, reliable, and very high speed links. Accordingly, the transmission delay and failure probability of exchanging the intermediate results among servers is negligible.

B. Sequential Offloading Scheme

In the sequential offloading scheme, the mobile device offloads the sub-tasks to the servers in a time-sequential manner via a shared channel. The scheduling problem of this offloading scheme consists of assigning sub-tasks to servers under the following constraints:

- Each sub-task can be offloaded to only one server.
- Each server can handle (receive or compute) only one sub-task at any time instant.
- If $x_{lj} = 1$, then the sub-task τ_j can be offloaded to a server; however, the server holds the computing of τ_j until the computation of τ_l is finished.

In the sequential task offloading, the uplink transmission of the j-th sub-task will not start until uplink offloading of all previous $(j-1)$ sub-tasks is finished. In other words, the waiting time of the j-th sub-task before transmission is $\mathcal{W}_j(\boldsymbol{\eta}) = \sum_{l=1}^{j-1} \mathcal{D}u_l(\boldsymbol{\eta})$, $2 \leq j \leq J$, and $\mathcal{W}_1(\boldsymbol{\eta}) = 0$. A sub-task τ_j $(j>1)$ may depend on one or more of the previous $(j-1)$ sub-tasks. Since the server holds the computing of τ_j until the computation of all sub-tasks with $x_{lj} = 1$ in $\mathbf{x}$ is finished, the additional holding delay of the j-th sub-task with sequential offloading can be expressed as:

$$\mathcal{D}h_j(\boldsymbol{\eta}) = \max_{\forall l<j} \left\{ x_{lj} \left[\mathcal{D}h_l(\boldsymbol{\eta}) + \mathcal{D}c_l(\boldsymbol{\eta}) - \sum_{r=l+1}^{j} \mathcal{D}u_r(\boldsymbol{\eta}) \right]^{+} \right\}. \tag{3}$$

The delay for offloading and computing the j-th sub-task can be expressed as

$$\mathcal{D}_j(\boldsymbol{\eta}) = \mathcal{W}_j(\boldsymbol{\eta}) + \mathcal{D}u_j(\boldsymbol{\eta}) + \mathcal{D}c_j(\boldsymbol{\eta}) + \mathcal{D}h_j(\boldsymbol{\eta}). \tag{4}$$

One server s_k is selected as a *sink server* to collect and send back the results to the mobile device. The total latency for offloading the task is given by

$$\mathcal{L}(\boldsymbol{\eta}) = \max_{j\in\mathcal{T}}\{\mathcal{D}_j(\boldsymbol{\eta})\} + \mathcal{D}_{D_k}, \tag{5}$$

where $\mathcal{D}_{D_k} = \frac{DN_p}{Rd_k}$ is the downlink transmission delay of the resulted data D by the sink server s_k.

The failure probability of the uplink transmission of task $\mathcal{T}$ can be written as

$$P^u(\boldsymbol{\eta}) = 1 - \prod_{j=1}^{J} \left(1 - P_j^u(\boldsymbol{\eta})\right), \tag{6}$$

where

$$P_j^u(\boldsymbol{\eta}) = 1 - \prod_{i=1}^{I} (1-p_i)^{\eta_{ij} u_j} \tag{7}$$

is the uplink transmission failure probability of offloading the j-th sub-task under the offloading decision $\boldsymbol{\eta}$. The failure probability of the downlink transmission of task $\mathcal{T}$ can be written as

$$P^d = 1 - (1-q_k)^D, \tag{8}$$

where the k-th server is selected as the sink server. Consequently, the task offloading failure probability with sequential offloading scheme can be expressed as

$$P(\boldsymbol{\eta}) = P^u(\boldsymbol{\eta}) + \left[1 - P^u(\boldsymbol{\eta})\right] P^d. \tag{9}$$

An illustration example is provided in the next section for ease of understanding.

III. Motivating Example

In this section, we discuss a numerical example to emphasize on the need for a good scheduling decision. Let us investigate a simple example in which the task $\mathcal{T}$ consists of four sub-tasks which can be represented as follows:
$\tau_1 = \{112, 0.85 \times 10^7, 23\}$; $\tau_2 = \{162, 2.48 \times 10^7, 33\}$;
$\tau_3 = \{589, 7.05 \times 10^7, 116\}$; $\tau_4 = \{137, 1.13 \times 10^7, 28\}$.
The sub-task dependency matrix associated with these sub-tasks is given as:

$$\mathbf{x} = \begin{array}{c} \begin{matrix} \tau_1 & \tau_2 & \tau_3 & \tau_4 \end{matrix} \\ \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array} \begin{matrix} \tau_1 \\ \tau_2 \\ \tau_3 \\ \tau_4 \end{matrix}. \tag{10}$$

The server set consists of four servers $\mathcal{S} = \{s_1, s_2, s_3, s_4\}$, represented as follows:
$s_1 = \{4.3\times10^6, 5.2\times10^6, 7.1\times10^9, 0.16\times10^{-6}, 1.0\times10^{-7}\}$;
$s_2 = \{3.9\times10^6, 4.8\times10^6, 2.0\times10^9, 7.18\times10^{-6}, 1.2\times10^{-7}\}$;
$s_3 = \{5.5\times10^6, 6.5\times10^6, 3.6\times10^9, 0.10\times10^{-6}, 0.3\times10^{-7}\}$;
$s_4 = \{6.4\times10^6, 6.4\times10^6, 7.2\times10^9, 0.07\times10^{-6}, 0.4\times10^{-7}\}$.

Let us assume that the offloading decision $\boldsymbol{\eta}$ is selected such that:

$$\boldsymbol{\eta} = \begin{array}{c} \begin{matrix} \tau_1 & \tau_2 & \tau_3 & \tau_4 \end{matrix} \\ \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \end{array} \begin{matrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{matrix}, \tag{11}$$

and s_4 is selected as the sink server. The total latency and offloading failure probability for the task are 54.1 (ms) and 1.25×10^{-3}, respectively. By examining all possible offloading decisions, the task can be offloaded in only 40.1 (ms) with the offloading failure probability of 8.2×10^{-5} with the following offloading decision:

$$\boldsymbol{\eta}^* = \begin{array}{c} \begin{matrix} \tau_1 & \tau_2 & \tau_3 & \tau_4 \end{matrix} \\ \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \end{array} \begin{matrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{matrix}, \tag{12}$$

and s_3 as the sink server. The offloading scheme of this preferable task offloading decision is shown in Fig. 1.

The question is how to find such a preferable offloading decision in a systematic way, for an arbitrary number of servers and sub-tasks. This motivates the problem formulation in the next section.

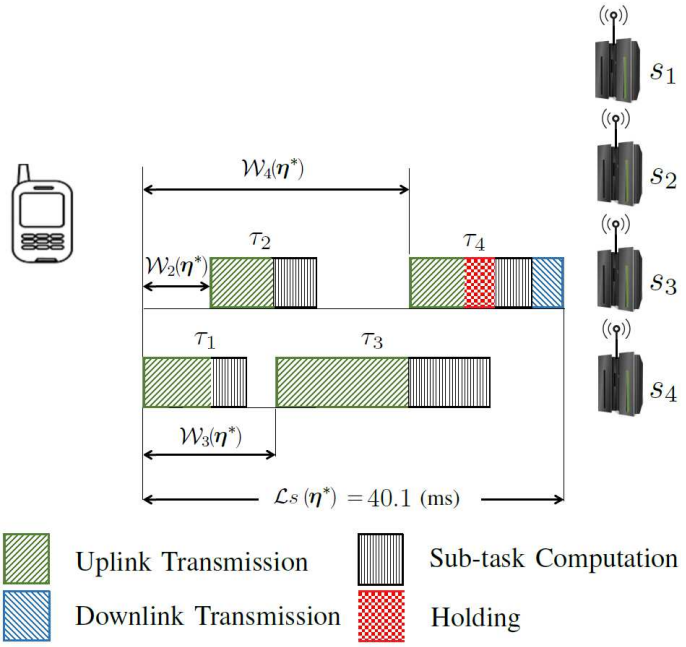


Figure 1: Sequential offloading scheme with offloading decision $\boldsymbol{\eta}^*$.

IV. Problem Formulation

Our objective is to minimize both the total latency for completing the task and the offloading failure probability simultaneously. To tackle the trade-off between latency and reliability and to give them a similar significance, we consider the weighted product method [11], in which the latency-reliability cost function can be formulated as[1]

$$\Psi(\boldsymbol{\eta}) = \mathcal{L}(\boldsymbol{\eta}) P(\boldsymbol{\eta}). \tag{13}$$

Consequently, the optimization problem is formulated as

$$\min_{\boldsymbol{\eta}} \quad \Psi(\boldsymbol{\eta}), \tag{14a}$$

$$\text{s.t.} \quad \sum_{i=1}^{I} \eta_{ij} = 1, \ \forall j \in \mathcal{T}, \tag{14b}$$

$$\eta_{i(j-1)} + \eta_{ij} \leq 1, \ \forall i \in \mathcal{S}, \ 2 \leq j \leq J, \tag{14c}$$

$$\eta_{ij}\eta_{il}\left[\sum_{r=j+1}^{l-1} \mathcal{D}u_r(\boldsymbol{\eta}) - \left(\mathcal{D}c_j(\boldsymbol{\eta}) + \mathcal{D}hs_j(\boldsymbol{\eta})\right)\right] \geq 0, \tag{14d}$$

$$\forall i \in \mathcal{S}, \ 1 \leq j \leq J-2, j+2 \leq l \leq J,$$

$$\eta_{ij} \in \{0, 1\}, \ \forall i \in \mathcal{S} \text{ and } j \in \mathcal{T}. \tag{14e}$$

Constraint (14b) guarantees that each sub-task is offloaded to only one server. Furthermore, constraints (14c) and (14d)

[1]For error-free environment, the cost function is $\Psi(\boldsymbol{\eta}) = \mathcal{L}(\boldsymbol{\eta})$.

guarantee that no two successive sub-tasks are offloaded to the same server and no sub-task is offloaded to a server that still handles another sub-task, respectively. The sink server s_k can be selected such that

$$s_k = \arg\min_{i \in \mathcal{S}} \{\mathcal{D}_{D_i} \left(1 - (1 - q_i)^D\right)\}, \tag{15}$$

and for error free environment, $s_k = \arg\min_{i \in \mathcal{S}} \{\mathcal{D}_{D_i}\}$.

It is worth noting that the search space of the optimization problem in (14) is 2^{IJ}. Consequently, the complexity of finding the optimum solution using exhaustive search is prohibitive for a reasonable number of servers and sub-tasks. The next section presents a more efficient method to solve the problem using GA.

V. Proposed Solution Using Genetic Algorithm

A genetic algorithm (GA) is a metaheuristic search technique, which evaluates a set of potential solutions called the initial population, each potential solution being referred to us chromosome. The GA generates new solutions through integrating the good features of existing solutions.

In the proposed GA, a potential offloading decision is represented as a set of J parameters known as genes g_i, each gene representing an association of a sub-task and a server. These genes are joined together to form a string of integers known as a chromosome $\mathbf{g}$. A chromosome (J-dimensional vector of integer numbers) identifies the servers, as assigned to the vector elements represented by the sub-tasks. For example, the offloading decision in (12) is represented by the following chromosome

$$\mathbf{g}^* = \begin{matrix} \tau_1 & \tau_2 & \tau_3 & \tau_4 \\ [\,4 & 3 & 4 & 3\,] \end{matrix}. \tag{16}$$

A. Feasibility and Fitness Evaluation

The chromosome representation ensures that constraints (14b) and (14e) are automatically satisfied since each sub-task is associated with exactly one server. However, this representation does not guarantee that constraints (14c) and (14d) are fulfilled. Algorithm 1 returns the feasibility indicator for a given chromosome $\mathbf{g}_k$ in the sequential offloading scheme. In this algorithm, if $\xi' = 0$ then $\mathbf{g}_k$ violates (14c) and if $\xi'' < 0$ then $\mathbf{g}_k$ violates (14d). A given chromosome is feasible if Algorithm 1 returns $\zeta_k = 1$.

We define the raw fitness of a chromosome $\mathbf{g}_k$ as

$$\theta_k = \frac{1}{\Psi(\boldsymbol{\eta}_k)}, \tag{17}$$

where $\boldsymbol{\eta}_k$ is the offloading decision associated with the chromosome $\mathbf{g}_k$. The fitness of a chromosome $\mathbf{g}_k$ is defined as

$$\Theta_k = \zeta_k \theta_k. \tag{18}$$

Hence, the chromosome with the highest $\Theta_k > 0$ is feasible and minimizes the objective function in (14).

Algorithm 1 Chromosome Feasibility for Sequential Offloading Scheme

input $\mathbf{g}_k$, du_{ij}, dc_{ij}, and $\mathbf{x}$
$\zeta_k = 1$
$\xi' = \prod_{j=1}^{J-1} \mid g_j - g_{j+1} \mid$
if $\xi' = 0$ **then**
  $\zeta_k \leftarrow 0$
  Return ζ_k
else
  Obtain $\boldsymbol{\eta}_k$ associates with $\mathbf{g}_k$
  $\boldsymbol{\eta} \leftarrow \boldsymbol{\eta}_k$.
  for $i = 1$ to I **do**
    for $j = 1$ to $J - 2$ **do**
      for $l = j + 2$ to J **do**
        $\xi'' = \eta_{ij}\eta_{il}\left[\sum_{r=j+1}^{l-1} \mathcal{D}u_r(\boldsymbol{\eta}) - \left(\mathcal{D}c_j(\boldsymbol{\eta}) + \mathcal{D}h_j(\boldsymbol{\eta})\right)\right]$
        if $\xi'' < 0$ **then**
          $\zeta_k \leftarrow 0$
          Return ζ_k
        end if
      end for
    end for
  end for
end if
Return ζ_k

B. Genetic Algorithm

The adopted GA comprise of the following steps:

1) Generate an initial population of Ξ randomly constructed chromosomes. Each chromosome in the initial population is generated by randomly associating a server with each sub-task.
2) Evaluate the fitness and the feasibility indicators of each chromosome. Two values are associated with each chromosome, namely the raw fitness value and the feasibility indicator [12].
3) Select two chromosomes as parents for reproduction. The binary tournament selection is adopted, in which two chromosomes are randomly chosen from the population and the one with the highest raw fitness value is considered as a parent [12]. The same procedure is applied to select the other parent.
4) Generate a child chromosome from the parents by first applying a *crossover* operation. One-point crossover operation is adopted, in which a crossover point $1 \leq j \leq J$ is randomly chosen. The child is structured from the first j genes which are taken from one of the parents, and from the remaining $J - j$ genes which are taken from the other parent. The crossover operation is followed by a *mutation* operation, in which two randomly selected genes in the child are exchanged (i.e., exchanging assigned servers between two randomly selected sub-tasks). After generating the child, the fitness and the feasibility indicator of the

child chromosome are evaluated.

5) Replace a chromosome in the population by the child chromosome. The adopted replacement procedure is as follows. The chromosome with zero feasibility indicator and lowest raw fitness value is replaced by the child. If all the chromosomes in the population are feasible, the chromosome with the lowest raw fitness value is replaced by the child.
6) Repeat steps 3 and 4 until N offspring chromosomes have been generated without enhancing the best chromosome found so far or a maximum number of offspring chromosomes M has been generated.

VI. Complexity Analysis

This section introduces the computational complexity of the GA. The basic operations performed in the GA algorithm are selection operation, crossover operation, mutation operation, chromosome replacement, and fitness evaluation. In the GA algorithm, generating a child chromosome requires the following operations. Parent chromosomes selection requires $\mathcal{O}(J)$, the crossover process has a complexity of $\mathcal{O}(J)$, the mutation process has also a complexity of $\mathcal{O}(J)$, and chromosome replacement requires $\mathcal{O}(\Xi)$ operations. Consequently, generating the offspring chromosomes requires $\mathcal{O}\left([J+\Xi]M\right)$ operations. To evaluate the fitness of a chromosome, the following operations are required. Calculating the holding time of each sub-task requires $\mathcal{O}(J)$ operations. Consequently, calculating the total latency requires $\mathcal{O}\left(J^2\right)$ operations. Calculating the offloading failure probability requires $\mathcal{O}(IJ)$ operations. The time complexity of finding the feasibility indicator ζ_k is $\mathcal{O}\left(IJ^2\right)$. Consequently, evaluating the fitness of a given chromosome has a time complexity of $\mathcal{O}\left(J^2+IJ+IJ^2\right) = \mathcal{O}\left(IJ^2\right)$ and the computational complexity of the GA with the sequential offloading scheme is $\mathcal{O}\left(IJ^2[\Xi+M]+[J+\Xi]M\right) = \mathcal{O}\left(IJ^2[\Xi+M]+\Xi M\right)$. It is worth mentioning that this is reduced when compared with the computational complexity of finding the optimal solution using the exhaustive search, $\mathcal{O}\left(2^{IJ}IJ^2\right)$.

VII. Simulation Results

In this section, we evaluate the performance of the proposed algorithm and study the effect of the main parameters on the latency and offloading failure probability. In obtaining these results, the servers are uniformly distributed with distances from the mobile device as 200 m $\sim$ 400 m. We set the channel gain as $h_i = \delta_i^{-\varepsilon}$, where δ_i is the distance between the mobile device and the i-th server and $\varepsilon = 4$ is the path loss exponent. The spectral density of the additive noise is $N_0 = -173$ dBm/Hz and the channel bandwidth is 1 MHz. The transmit power of the mobile device and the servers is 100 mW. The remaining parameters are as follows. The computation speed of the MEC server is generated randomly from a uniform distribution ranging from 2×10^9 to 8×10^9 cycles per second [7], the packet size is $N_p = 20$ bytes [13], task input and output data are $U = 1000$ packets and $D = 200$ packets, respectively, and the computational complexity of a sub-task α_j in cycles per bit follows Gamma distribution with shape and scale parameter of 4 and 200, respectively [14]. To represent the dependency among the sub-tasks, a given sub-task depends on any of the previous sub-tasks with probability $\pi = 0.3$. The initial population size is $10IJ$ chromosomes [15], and N and M equal 10^3 and 10^5, respectively.

Figure 2 illustrates the performance of the GA and optimum solution (obtained using exhaustive search) versus the task data size U. It is seen that the proposed GA solution achieves near-optimum performance. It is clear that there is no remarkable change in the number of contributing servers versus the task data size U.

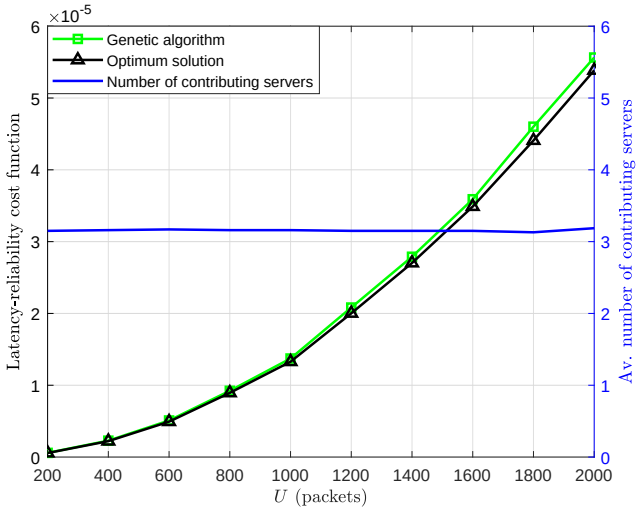


Figure 2: Latency-reliability cost function versus the task data size U.

In Fig. 3, all the servers are located at the same distance from the mobile device (i.e., $\delta_i = \delta\ \forall s_i \in \mathcal{S}$). Such scenario is suitable for the case where the servers are located in close proximity to each other or they are placed at the arc of a circle whose center is the mobile device. It is noticed that as the distance between the servers and the mobile device increases (i.e., the SNR decreases), the latency increases. This can be attributed to the fact that reducing the SNR will increase the uplink transmission delay, and consequently, the waiting time of the sub-tasks[2] also increases. It is also noticed that as the distance between the servers and the mobile device increases (i.e., uplink transmission delay increases), the number of contributed servers decreases. This is because during the long time duration of uplink transmission of a sub-task, more servers will finish the computation of their sub-task and be ready to contribute. Such availability of servers gives the scheduler the ability to offload more sub-tasks to less number of servers (because it selects the best available servers).

[2]The waiting time of a sub-task is an accumulation of the uplink delay of all previous sub-tasks, i.e., $\mathcal{W}_j(\boldsymbol{\eta}) = \sum_{l=1}^{j-1} \mathcal{D}u_l(\boldsymbol{\eta})$.

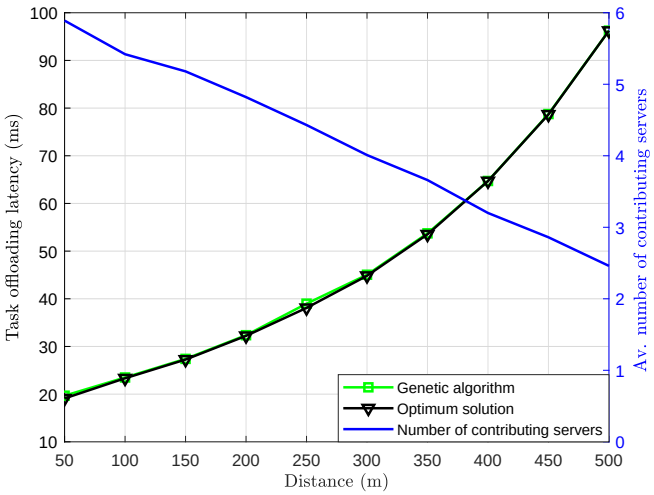


Figure 3: Latency versus distance with $I = 6$ servers and $J = 6$ sub-tasks.

VIII. Conclusion

A task scheduling was proposed for sequential offloading of a delay-sensitive and computationally-intensive task to multiple MEC servers. The problem was formulated as a joint optimization of latency and offloading failure probability. A heuristic solution was developed based on a genetic algorithm. Simulation results revealed that the proposed solution provides a performance close to the optimal one, with reduced complexity.

Acknowledgment

This work was supported in part by the Natural Science and Engineering Research Council of Canada (NSERC) through its Discovery Program, the Chair of Excellence Program and Spanish National Project TERESA-ADA (TEC2017-90093-C3-2-R) (MINECO/AEI/FEDER, UE).

References

[1] A. U. R. Khan, M. Othman, S. A. Madani, and S. U. Khan, "A survey of mobile cloud computing application models," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 1, pp. 393–413, First quarter 2014.

[2] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Fourth quarter 2017.

[3] N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, "Mobile edge computing: A survey," *IEEE Internet Things J.*, vol. 5, no. 1, pp. 450–465, Feb. 2018.

[4] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, Third quarter 2017.

[5] J. Ren, G. Yu, Y. Cai, Y. He, and F. Qu, "Partial offloading for latency minimization in mobile-edge computing," in *Proc. IEEE Global Commun. Conf.*, Dec. 2017, pp. 1–6.

[6] S. Bi and Y. J. Zhang, "Computation rate maximization for wireless powered mobile-edge computing with binary computation offloading," *IEEE Trans. Wireless Commun.*, vol. 17, no. 6, pp. 4177–4190, Jun. 2018.

[7] T. Q. Dinh, J. Tang, Q. D. La, and T. Q. S. Quek, "Offloading in mobile edge computing: Task allocation and computational frequency scaling," *IEEE Trans. Commun.*, vol. 65, no. 8, pp. 3571–3584, Aug. 2017.

[8] J. Liu and Q. Zhang, "Offloading schemes in mobile edge computing for ultra-reliable low latency communications," *IEEE Access*, vol. 6, pp. 12 825–12 837, Feb. 2018.

[9] Y. Wang, M. Sheng, X. Wang, L. Wang, and J. Li, "Mobile-edge computing: Partial computation offloading using dynamic voltage scaling," *IEEE Trans. Commun.*, vol. 64, no. 10, pp. 4268–4282, Oct. 2016.

[10] S. E. Mahmoodi, R. N. Uma, and K. P. Subbalakshmi, "Optimal joint scheduling and cloud offloading for mobile applications," *IEEE Trans. Cloud Comput.*, vol. 7, no. 2, pp. 301–313, Apr. 2019.

[11] R. Marler and J. Arora, "Survey of multi-objective optimization methods for engineering," *Structural and Multidisciplinary Optimization*, vol. 26, no. 6, pp. 369–395, Apr. 2004.

[12] P. C. Chu and J. E. Beasley, "A genetic algorithm for the generalised assignment problem," *Computers & Operations Research*, vol. 24, no. 1, pp. 17–23, Apr. 1997.

[13] C. She, C. Yang, and T. Q. S. Quek, "Cross-layer optimization for ultra-reliable and low-latency radio access networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 1, pp. 127–141, Jan. 2018.

[14] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, and D. O. Wu, "Energy-optimal mobile cloud computing under stochastic wireless channel," *IEEE Trans. Wireless Commun.*, vol. 12, no. 9, pp. 4569–4581, Sep. 2013.

[15] R. Storn, "On the usage of differential evolution for function optimization," in *Proc. IEEE North American Fuzzy Information Processing*, 1996, pp. 519–523.